

Day 12 — Jobs & Pipelines

Status: ■ Not started

Objectives

- Understand Freestyle vs Pipeline jobs
- Write a Declarative Jenkinsfile with multiple stages
- Use parameters, environment variables, and post-build actions
- Understand Scripted pipeline basics for comparison

Lab Steps

Create a Pipeline job

- New Item → Pipeline named pipeline-basics
- Use the jenkinsfiles/basic/Jenkinsfile from this repo as a starting point

Add parameters

- Add a choice parameter for environment (staging/production)
- Add a boolean parameter to toggle running tests

Add environment variables

- Set APP_ENV from the parameter
- Reference it inside a shell step

Post-build actions

- Add post { success {} failure {} always {} } blocks
- Archive a dummy artifact

Scripted pipeline comparison

- Rewrite the same pipeline using node { } + stage() scripted syntax
- Note the differences in error handling (try/catch vs post)

Key Takeaways

- Declarative pipelines are linted before running — catches syntax errors early
- when blocks control conditional stage execution (branch, expression, environment)
- Parameters make jobs reusable across environments

Checklist

- [] Pipeline job created from Jenkinsfile in SCM

- [] Parameters and environment variables working
- [] Post-build success/failure/always blocks tested
- [] Scripted pipeline equivalent written for comparison

Day 12 — Interview Questions: Jobs & Pipelines

What's the difference between Declarative and Scripted pipeline syntax?

Why are Declarative pipelines validated before any stage runs, and what's the practical benefit?

How do you pass parameters into a pipeline, and how do you access them in stages?

Explain the `environment { }` block — when do those variables become available?

What does the `when` directive do? Give three example conditions.

How do you run stages in parallel, and when would you do so?

What's the purpose of the `post { }` block, and what conditions can it react to (success, failure, always, etc.)?

How would you retry a flaky step without failing the whole pipeline?

What does `agent any` vs `agent { docker { image '...' } }` change about where a stage runs?

How do Shared Libraries help avoid duplicated pipeline code across projects?

<details><summary>Answer notes</summary>

- Q1: Declarative is structured/opinionated and linted upfront; Scripted is raw Groovy inside `node { }`, more flexible but errors surface at runtime.
- Q6: Use a `parallel { }` block inside a stage — useful for independent tasks like lint + unit tests to cut wall-clock time.
- Q8: Wrap the step in `retry(n) { ... }`.

</details>